

ACE-PruningRAG

Adaptive Cross-Source Evidence Pruning for Multi-Source
RAG

5-Phase Research + Implementation Roadmap

Base Paper

**Multi-Source Knowledge Pruning for Retrieval-Augmented Generation:
A Benchmark and Empirical Study**

Shuo Yu et al. | Accepted at CIKM 2025

[Official GitHub: github.com/USTCAGI/PruningRAG](https://github.com/USTCAGI/PruningRAG)

[Paper: arxiv.org/abs/2409.13694](https://arxiv.org/abs/2409.13694)

[Dataset: huggingface.co/datasets/fishsure/RM3QA](https://huggingface.co/datasets/fishsure/RM3QA)

Objective: reproduce the published system, add a focused cross-source evidence contribution, validate it rigorously, and finish with both a conference-ready research package and a strong AI/ML resume project.

Project Overview

Project Name: ACE-PruningRAG - Adaptive Cross-Source Evidence Pruning for Multi-Source Retrieval-Augmented Generation

Base Paper

Title: Multi-Source Knowledge Pruning for Retrieval-Augmented Generation: A Benchmark and Empirical Study

Venue: CIKM 2025 (accepted)

Core idea: PruningRAG uses multi-granularity pruning to select relevant knowledge from multiple sources while reducing misleading or noisy context. The released system works with structured and unstructured knowledge, including web pages and mock APIs, and provides the RM3QA benchmark.

Official code: <https://github.com/USTCAGI/PruningRAG>

Paper: <https://arxiv.org/abs/2409.13694>

Dataset: <https://huggingface.co/datasets/fishsure/RM3QA>

What ACE-PruningRAG Changes

- Preserve the original PruningRAG pipeline as the main published baseline.
- Add adaptive multi-source retrieval so the system chooses which sources to use and how much evidence to retrieve.
- Move from independent relevance ranking to joint cross-source evidence selection, emphasizing complementary facts needed together to answer a query.
- Add evidence consistency, sufficiency, and claim-level verification to reduce unsupported answers.
- Evaluate both answer quality and efficiency under controlled retrieval budgets.

Main research question: Can jointly selecting complementary evidence across heterogeneous sources improve evidence completeness and grounded answer quality while reducing unnecessary retrieval?

Final System at a Glance

```

User Query -> Query Understanding
      |
Adaptive Source Router + Dynamic Retrieval Budget
      |
Web / API / Structured Data Retrievers
      |
Unified Evidence Pool -> Complementarity + Consistency Selection
      |
Sufficiency Check -- insufficient --> Targeted Re-retrieval
      |
Grounded Generation -> Claim Verification + Source Citations
```

PHASE 1 | REPRODUCE PRUNINGRAG EXACTLY

Goal: Establish a trustworthy, repeatable baseline before changing the research system.

1.1 Repository and Environment

- Fork or clone the official USTCAGI/PruningRAG repository and preserve the original main branch as an untouched baseline.
- Create a reproducible Python environment from requirements.txt; pin versions that change during setup.
- Download the required retriever/reranker models and configure either API-based or local LLM inference.
- Download and validate the RM3QA dataset; document any preprocessing or dataset-loader fixes needed for your environment.
- Record hardware, model versions, random seeds, prompts, and runtime configuration.

1.2 Reproduce the Original Pipeline

- Run the Domain Router and Dynamism Router exactly as released.
- Reproduce web retrieval: broad BM25 retrieval -> dense retrieval with BGE-M3 -> reranking with BGE reranker.
- Reproduce mock-API processing: named-entity extraction, entity matching, time extraction, API selection, and JSON-to-text/Markdown conversion.
- Run the original generation and evaluation flow without architectural modifications.
- Save per-query retrieval traces and final predictions so later improvements can be compared query-by-query.

1.3 Baseline Measurements

- Answer accuracy / exact-match-style metrics used by the base code
- Retrieval Recall@K and MRR where ground-truth retrieval information is available
- Hallucination / unsupported-answer behavior
- Average retrieved chunks and tokens per query
- Latency per query and number of LLM calls
- GPU memory or API cost where relevant

Phase 1 deliverable: A reproducible PruningRAG baseline with environment files, experiment configuration, baseline_results.csv, retrieval traces, and a short reproduction report.

PHASE 2 | ADAPTIVE MULTI-SOURCE RETRIEVAL

Goal: Improve where the system retrieves from and how much evidence each query receives.

2.1 Adaptive Multi-Label Source Router

- Represent source selection as a multi-label decision rather than a rigid single-source choice.
- For every query q , estimate a probability $P(S_i | q)$ for each source S_i .
- Use source probability thresholds to activate one or more sources; preserve the original routers as baselines.
- Start with a lightweight classifier or embedding-based classifier before trying a large LLM router.
- Log routing probabilities, selected sources, and downstream retrieval results for every query.

$P(S_i | q) \rightarrow$ choose source(s) only when their estimated utility is high enough

2.2 Query-Adaptive Retrieval Budget

- Replace fixed Top-K values with query-dependent retrieval budgets.
- Estimate query complexity and allocate K_{web} , K_{api} , $K_{structured}$, ... separately.
- Simple fact queries should retrieve very little; cross-source or multi-hop queries should receive a larger budget.
- Enforce an upper budget so accuracy improvements are not obtained merely by retrieving more context.
- Track tokens, retrieval time, and LLM calls to quantify the quality-efficiency trade-off.

$K_i = f(q, S_i, \text{complexity}, \text{budget})$

2.3 Experiments for Phase 2

- Original PruningRAG vs Adaptive Router.
- Original fixed Top-K vs Dynamic Top-K.
- Adaptive Router alone vs Adaptive Router + Dynamic Budget.
- Compare routing accuracy, downstream answer accuracy, retrieval recall, tokens/query, and latency.
- Include an always-search-all-sources baseline to show whether adaptive routing is actually useful.

Phase 2 deliverable: A modular `adaptive_router` component, `retrieval_budget` policy, query-complexity logic, and controlled experiments showing whether adaptive retrieval improves the quality-cost trade-off.

PHASE 3 | CROSS-SOURCE EVIDENCE INTELLIGENCE

Goal: Make joint complementary evidence selection the primary research contribution.

3.1 Unified Evidence Representation

- Normalize evidence from web text, API responses, and other structured sources into a common evidence object.
- Store source ID, entity, time, extracted fact/value, retrieval score, provenance, and raw supporting content.
- Maintain provenance through every stage so final claims can be traced back to exact evidence.

3.2 Complementary Evidence Selection

- Do not score every item only by independent relevance to the query.
- Select evidence sets that jointly cover the information needed to answer the question.
- Reward evidence that adds missing entities, dates, attributes, or comparison values rather than repeating already-retrieved facts.
- Penalize redundant evidence and excessive context cost.

Score(E) = α *Relevance + β *Coverage + γ *Complementarity + δ *Agreement - λ *Cost

Example: A 2023 value from a web page and a 2022 value from an API may each be only moderately relevant alone, but together they are sufficient for a year-over-year comparison. The selector should preserve both.

3.3 Evidence Graph

- Represent retrieved evidence as nodes in a lightweight graph (begin with NetworkX; use Neo4j only if scale later requires it).
- Create relations such as same_entity, same_event, same_time, different_time, supports, contradicts, and depends_on.
- Use graph features or pairwise scores to estimate whether evidence is complementary, redundant, or contradictory.
- Select a compact evidence subgraph for generation instead of blindly concatenating top-ranked chunks.

3.4 Cross-Source Consistency and Conflict Checking

- Before generation, compare candidate facts from different sources for value, entity, units, date, and definition compatibility.
- Distinguish genuine contradiction from different reporting periods or definitions.
- Flag unresolved conflicts so the generator can qualify the answer, select a newer/authoritative source, or abstain.

Phase 3 deliverable: A unified evidence pool + complementary evidence selector + evidence graph + conflict module. This phase should contain the central method you position as the paper contribution.

PHASE 4 | EVIDENCE SUFFICIENCY, VERIFICATION, AND RESEARCH EVALUATION

Goal: Make the system know when evidence is enough, when to retrieve again, and how to measure reliability.

4.1 Evidence Sufficiency Controller

- Estimate $P(\text{sufficient} \mid q, E)$ after cross-source evidence selection.
- If evidence is sufficient, proceed to generation.
- If evidence is insufficient, identify missing information and trigger targeted query reformulation/retrieval.
- Set a maximum number of retrieval rounds to prevent uncontrolled agentic loops.
- Measure whether iterative retrieval adds useful evidence or merely increases cost.

If $P(\text{sufficient} \mid q, E) \geq \text{threshold}$ -> Generate; otherwise -> Retrieve missing evidence

4.2 Claim-Level Verification

- Break the generated answer into atomic factual claims.
- Link each claim to one or more supporting evidence items.
- Classify claims as supported, partially supported, unsupported, or contradicted.
- For unsupported claims: remove them, retrieve again, or explicitly abstain.
- Expose source citations in the final demo so users can inspect the evidence path.

4.3 Build a Hard Cross-Source Evaluation Extension

- Create a carefully verified evaluation subset instead of a huge weak synthetic dataset.
 - Single-source questions - sanity check that the new method does not harm easy cases.
 - True cross-source questions - answer requires evidence from two or more different sources.
 - Distractor-source questions - irrelevant but highly similar evidence is present.
 - Missing-evidence questions - correct behavior may be abstention.
 - Conflict questions - sources provide incompatible claims.
 - Temporal-conflict questions - old and newer facts coexist.
- Target roughly 200-500 high-quality questions if manual verification resources permit; quality matters more than volume.

Phase 4 - Metrics and Evaluation Protocol

4.4 Metrics

Area	Metrics
Retrieval	Recall@K, Precision@K, MRR / nDCG where appropriate
Evidence completeness	CompleteEvidence@K / supporting-evidence coverage
Generation	Accuracy, Exact Match, F1, task-specific correctness
Grounding	Claim support rate, citation precision/recall, unsupported-claim rate
Reliability	Abstention precision, false-answer rate, conflict handling
Efficiency	Tokens/query, retrieved chunks/query, LLM calls, latency, GPU/API cost

$$\text{CompleteEvidence@K} = (\# \text{ queries where every required evidence item is retrieved}) / N$$

4.5 Controlled Evaluation Protocol

- Hold the generator model, corpus, prompt format, and maximum retrieval budget constant whenever comparing retrieval methods.
- Evaluate single-source and true cross-source questions separately so aggregate scores do not hide the main research effect.
- Report both end-task answer quality and evidence-level metrics; a correct answer with unsupported reasoning should not count as fully successful.
- Cache deterministic retrieval outputs where appropriate to reduce cost and make ablations reproducible.
- Perform manual error analysis on representative failures: missed evidence, redundant evidence, conflicts, premature sufficiency, and verifier mistakes.

Phase 4 deliverable: An iterative sufficiency-controlled pipeline, claim verifier, hard cross-source evaluation subset, and a metric suite that measures both correctness and evidence quality.

PHASE 5 | ABLATIONS, PUBLICATION PACKAGE, AND RESUME-GRADE DEMO

Goal: Prove the contribution scientifically, quantify trade-offs, then package the work as both a paper and a production-style project.

5.1 Required Baselines

- Vanilla retrieval-augmented generation baseline.
- BM25-only RAG.
- Dense-retrieval RAG.
- Original PruningRAG - primary published baseline.
- Always-retrieve-from-all-sources baseline.
- Adaptive Router only.
- Adaptive Router + Dynamic Budget.
- Cross-source evidence selector without sufficiency loop.
- Full ACE-PruningRAG.

5.2 Ablation Study

- Start from the full system and remove one component at a time.
 - Remove Adaptive source routing and measure the change.
 - Remove Dynamic retrieval budget and measure the change.
 - Remove Complementarity scoring and measure the change.
 - Remove Evidence graph features and measure the change.
 - Remove Conflict detection and measure the change.
 - Remove Evidence sufficiency loop and measure the change.
 - Remove Claim verification and measure the change.
- The paper should explicitly state which component contributes to which metric; avoid attributing all gains to the full system.

5.3 Efficiency and Statistical Validation

- Report answer quality together with retrieved context, latency, LLM calls, tokens, and monetary/compute cost.
- Use the same generator, corpus, prompts, and retrieval budget when comparing retrieval methods whenever possible.
- Run repeated experiments where randomness matters and report confidence intervals or paired significance tests for main comparisons.
- Perform qualitative error analysis on both improved and degraded examples.

5.4 Resume-Grade Application

- Backend: FastAPI with separate router, retriever, evidence, verifier, and generation services/modules.
- Search: BM25 + FAISS/pgvector for text; DuckDB for local structured data; NetworkX for evidence relationships initially.
- Frontend: Next.js interface showing sources selected, retrieved evidence, evidence relationships, final answer, and claim-level citations.
- Add experiment/config files so the research results are fully reproducible from the same repository.

5.5 Paper Positioning

Suggested paper title: Adaptive Cross-Source Evidence Selection for Efficient Multi-Source Retrieval-Augmented Generation

- Primary claim: joint complementary evidence selection improves cross-source evidence completeness compared with independent relevance pruning under a controlled retrieval budget.
- Secondary claim: adaptive retrieval budgets can reduce context/compute cost while retaining answer quality.
- Reliability contribution: sufficiency and claim verification reduce unsupported responses, especially on missing/conflicting evidence cases.
- Do not claim publication until experiments support the hypotheses; venue fit depends on final novelty and empirical strength.

Phase 5 deliverable: Conference-style manuscript + reproducible experiment repository + trained/configured components + evaluation artifacts + polished demo + resume-ready measured results.

End-to-End Execution Checklist

- 01 Fork PruningRAG and lock the untouched baseline.
- 02 Set up RM3QA, models, inference backend, and reproducible environment.
- 03 Reproduce baseline results and collect per-query traces.
- 04 Implement adaptive multi-label source routing.
- 05 Implement dynamic per-source retrieval budgets.
- 06 Normalize all retrieved items into a unified evidence schema.
- 07 Implement cross-source complementarity and redundancy scoring.
- 08 Add evidence graph relations and conflict detection.
- 09 Implement evidence sufficiency and targeted re-retrieval.
- 10 Add claim-level verification and citation tracing.
- 11 Create the hard cross-source evaluation subset.
- 12 Run all baselines, ablations, efficiency tests, and statistical analysis.
- 13 Write the paper around one central contribution, not a feature list.
- 14 Build the polished demo after the research pipeline is stable.
- 15 Replace resume claims with measured results only after experiments are complete.

What Success Looks Like

Research Output

- A clean reproduction of a published CIKM 2025 multi-source RAG system.
- A focused method for joint, budget-aware cross-source evidence selection.
- A hard evaluation set that tests complementary, missing, conflicting, and time-sensitive evidence.
- Ablation results showing exactly where gains come from.
- Quality-vs-cost analysis suitable for an IR/NLP/RAG paper.

Resume Output

Example resume positioning (replace placeholders with measured numbers): Extended CIKM 2025 PruningRAG with adaptive source routing, dynamic retrieval budgets, cross-source complementary evidence fusion, sufficiency checks, and claim-level verification; improved [metric] by [X] while reducing [tokens/latency/cost] by [Y] on RM3QA and a hard cross-source evaluation set.

Core Technical Skills Demonstrated

- Retrieval systems: BM25, dense embeddings, reranking
- LLM-based RAG and structured/API retrieval
- Multi-source routing and budget allocation
- Evidence graphs and cross-source reasoning
- Grounding, hallucination analysis, and verification
- Evaluation design, ablations, statistical testing, and reproducibility
- FastAPI/Next.js production-style ML system integration

Base Resources and Verified Links

Base paper

Shuo Yu et al. “Multi-Source Knowledge Pruning for Retrieval-Augmented Generation: A Benchmark and Empirical Study.” Accepted at CIKM 2025.

<https://arxiv.org/abs/2409.13694>

Official implementation

<https://github.com/USTCAGI/PruningRAG>

RM3QA dataset

<https://huggingface.co/datasets/fishsure/RM3QA>

Verification note: Paper title, CIKM 2025 acceptance, official code link, and released multi-source RM3QA dataset were checked against the current arXiv record and official GitHub repository on October 7, 2026. The roadmap above is a proposed extension plan; it does not claim that the proposed method has already been experimentally validated or accepted for publication.